

IU INTERNATIONALE HOCHSCHULE | PORTFOLIO-ARBEIT

Phase 2: Reflexions- und Entwurfsbericht zur Softwarearchitektur

Modul:	Objektorientierte und funktionale Programmierung mit Python (DLBDSOOFPP01_D)
Studierender:	Manuel Hanack (Matrikelnummer: IU14156336)
Studiengang:	B.Sc. Cyber Security
Dokument	Reflexions- und Entwurfsbericht (Phase 2)
Abgabedatum:	06.10.2026

1. Untersuchung und Diskussion der Umsetzung objektorientierter Konzepte in Python

Der Übergang von einem sprachunabhängigen UML-Entity-Klassendiagramm aus Phase 1 hin zu einer konkreten Implementierung in Python offenbart einige sprachspezifische Besonderheiten. Im Folgenden wird detailliert reflektiert, wie die Kernkonzepte der Objektorientierung in Python umgesetzt werden:

1. Kapselung (Encapsulation) & Datenvalidierung via Properties

Python bietet auf Sprachebene kein hartes 'private'-Schlüsselwort. Stattdessen basiert Kapselung auf Konventionen (`_attribute` für geschützte Zugriffe) und dem Konzept von Properties (`@property` und `@setter`). In der Klasse `Pruefungsleistung` wird die Kapselung genutzt, um Noteneingaben aktiv zu validieren:

- **Notenvalidierung:** Der Setter für `note` akzeptiert ausschließlich Werte der offiziellen IU-Notenskala (1.0, 1.3, 1.7, 2.0, 2.3, 2.7, 3.0, 3.3, 3.7, 4.0 sowie 5.0 bei Fehlversuchen). Ungültige Werte lösen eine kontrollierte `ValueError`-Exception aus.
- **Abgeleitetes Attribut (/bestanden):** Das im UML-Diagramm als abgeleitet gekennzeichnete Feld `/bestanden` wird im Setter automatisch berechnet (`self._bestanden = value <= 4.0`). Dadurch bleibt der Objektzustand zu jedem Zeitpunkt konsistent.

2. Aufzählungs-Typen (Enumerations) für ModulStatus

In Phase 1 wurde der Status eines Moduls als Freitext gehandhabt. Gemäß dem Tutor-Feedback wurde in Phase 2 eine explizite Enumeration `ModulStatus` über das Python-Modul `enum` eingeführt. Die Zustände `OFFEN`, `IN_BEARBEITUNG` und `BESTANDEN` verhindern Tippfehler und ermöglichen ein typsicheres Mapping zwischen Benutzeroberfläche, Fachmodell und relationaler SQLite-Datenbank.

3. Komposition vs. Aggregation in der Lebenszyklussteuerung

Da Python Variablen stets als Objektreferenzen verwaltet, unterscheidet sich die technische Umsetzung von Komposition und Aggregation nicht im Speicherlayout, sondern rein in der logischen Lebenszyklussteuerung:

- **Komposition (Studiengang → Semester):** Die Klasse Studiengang erzeugt und verwaltet ihre Semester-Instanzen direkt. Beim Löschen oder Zurücksetzen des Studiengangs verfallen auch die enthaltenen Semester-Objekte.
- **Aggregation (Semester → Modul):** Module existieren als eigenständige Entitäten im Fachmodell. Ein Modul kann im Studienverlauf zwischen Semestern verschoben werden, ohne dass das Modul-Objekt selbst zerstört wird.

2. Gesamtarchitektur & UML-Architekturdiagramm (MVC-Muster)

Um eine saubere Trennung der Belange zu gewährleisten, folgt die Gesamtarchitektur des Dashboard-Prototypen streng dem Model-View-Controller (MVC) Architekturmuster:

- **Model-Schicht (models.py):** Enthält die reinen Business-Klassen (Studiengang, Semester, Modul, Pruefungsleistung, ModulStatus). Diese Schicht ist komplett frei von GUI- oder Datenbank-Code und kapselt die Rechen- und Validierungslogik (Notenschnitt, ECTS-Fortschritt).
- **Data Access Layer (database.py):** Die Klasse DatabaseManager verwaltet die relationale SQLite-Datenbank (dashboard.db). Sie erstellt beim Erststart automatisch das Tabellenschema module_progress und befüllt es mit den 29 Modulen.
- **View-Schicht (gui.py):** Implementiert die grafische Benutzeroberfläche auf Basis von Tkinter und ttk. Die Klasse StudentDashboardApp visualisiert die KPI-Karten, den Fortschrittsbalken, Filter-Dropdowns sowie die Modultabelle und stellt Eingabemasken bereit.
- **Einstiegspunkt (main.py):** Fungiert als Bootstrapper. Initialisiert den DatabaseManager, verknüpft die Komponenten und startet die Tkinter-Eventloop.

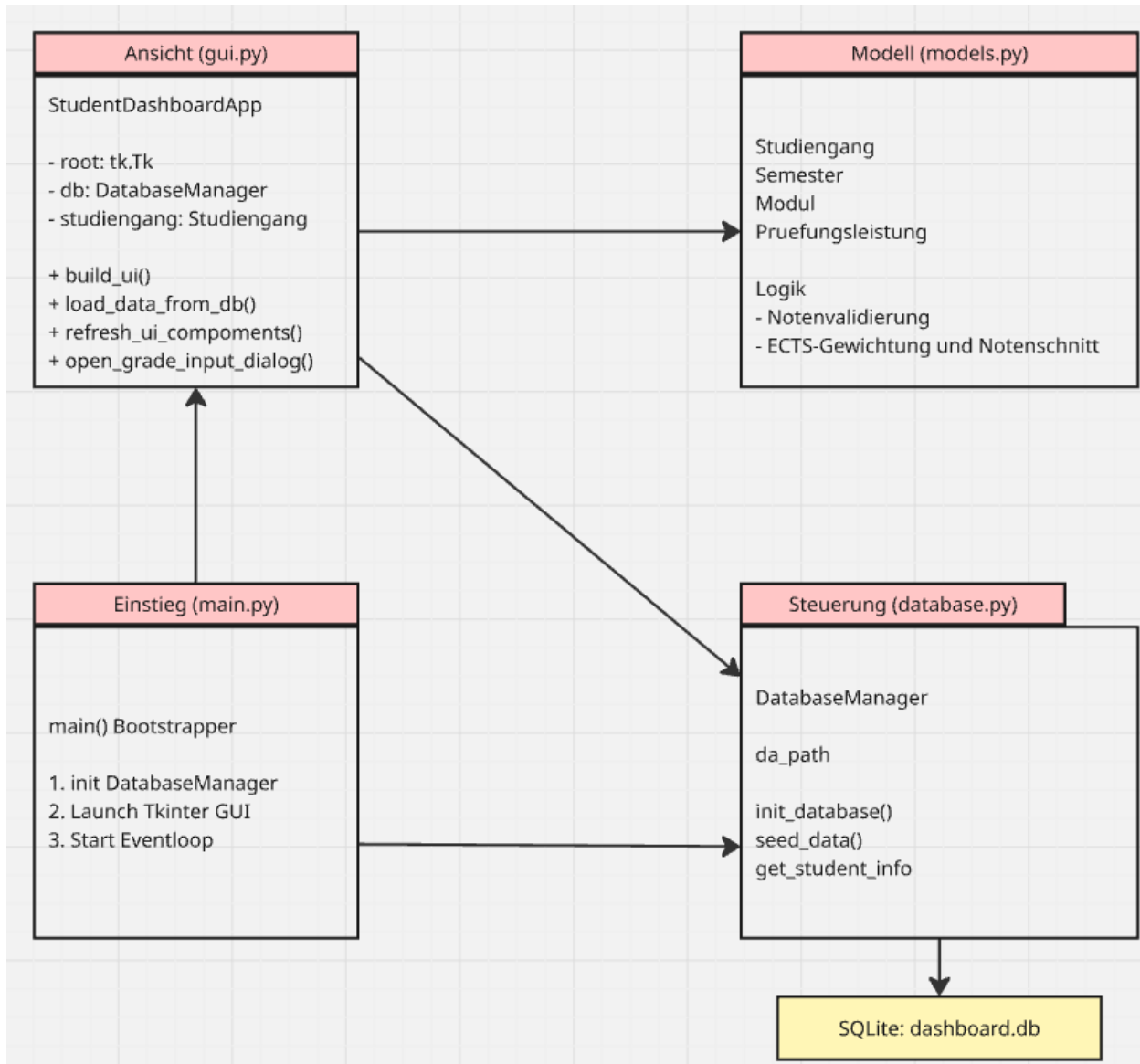


Abbildung 3: UML-Systemarchitekturdiagramm (MVC-Muster) mit direkter GUI-Anbindung